

```
In [4]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
```

```
In [40]: df = pd.read_csv("Zomato data .csv")
df
```

Out[40]:

	name	online_order	book_table	rate	votes	approx_cost(for two people)	listed_in(type of cuisine)
0	Jalsa	Yes	Yes	4.1/5	775	800	Buffet
1	Spice Elephant	Yes	No	4.1/5	787	800	Buffet
2	San Churro Cafe	Yes	No	3.8/5	918	800	Buffet
3	Addhuri Udupi Bhojana	No	No	3.7/5	88	300	Buffet
4	Grand Village	No	No	3.8/5	166	600	Buffet
...	...	...	...	...	...	...	...
143	Melting Melodies	No	No	3.3/5	0	100	Dinner
144	New Indraprasta	No	No	3.3/5	0	150	Dinner
145	Anna Kuteera	Yes	No	4.0/5	771	450	Dinner
146	Darbar	No	No	3.0/5	98	800	Dinner
147	Vijayalakshmi	Yes	No	3.9/5	47	200	Dinner

148 rows × 7 columns



```
In [42]: df.head()
```

Out[42]:

	name	online_order	book_table	rate	votes	approx_cost(for two people)	listed_in(type)
0	Jalsa	Yes	Yes	4.1/5	775	800	Buffet
1	Spice Elephant	Yes	No	4.1/5	787	800	Buffet
2	San Churro Cafe	Yes	No	3.8/5	918	800	Buffet
3	Addhuri Udupi Bhojana	No	No	3.7/5	88	300	Buffet
4	Grand Village	No	No	3.8/5	166	600	Buffet

In [43]: `print("Shape:", df.shape)`

Shape: (148, 7)

In [44]: `print("Columns:")`
`print(df.columns)`

Columns:

Index(['name', 'online_order', 'book_table', 'rate', 'votes',
'approx_cost(for two people)', 'listed_in(type)'],
dtype='object')In [45]: `print("Data Types:")`
`print(df.dtypes)`

Data Types:

name	object
online_order	object
book_table	object
rate	object
votes	int64
approx_cost(for two people)	int64
listed_in(type)	object

dtype: object
In [46]: `print("Missing Values:")`
`print(df.isnull().sum())`

Missing Values:

name	0
online_order	0
book_table	0
rate	0
votes	0
approx_cost(for two people)	0
listed_in(type)	0

dtype: int64
In [47]: `print("Duplicate Records:", df.duplicated().sum())`

Duplicate Records: 0

In [48]: `print(df.describe())`

	votes	approx_cost(for two people)
count	148.000000	148.000000
mean	264.810811	418.243243
std	653.676951	223.085098
min	0.000000	100.000000
25%	6.750000	200.000000
50%	43.500000	400.000000
75%	221.750000	600.000000
max	4884.000000	950.000000

In [49]: `df = df.drop_duplicates()`

```

for col in df.select_dtypes(include=np.number):
    df[col] = df[col].fillna(df[col].median())

for col in df.select_dtypes(include='object'):
    df[col] = df[col].fillna(df[col].mode()[0])

print("Missing values and duplicates handled.")

```

Missing values and duplicates handled.

In [50]: `df['rate'] = df['rate'].astype(str).str.replace('/5', '', regex=False)`
`df['rate'] = pd.to_numeric(df['rate'], errors='coerce')`

```

df['approx_cost(for two people)'] = (
    df['approx_cost(for two people)']
    .astype(str)
    .str.replace(',', '', regex=False)
)

df['approx_cost(for two people)'] = pd.to_numeric(
    df['approx_cost(for two people)'], errors='coerce'
)

print(df[['rate', 'approx_cost(for two people)']].head())

```

	rate	approx_cost(for two people)
0	4.1	800
1	4.1	800
2	3.8	800
3	3.7	300
4	3.8	600

In [51]: `df = df[(df['rate'] >= 0) & (df['rate'] <= 5)]`

```

df['online_order'] = df['online_order'].str.strip().str.title()
df['book_table'] = df['book_table'].str.strip().str.title()
df['listed_in(type)'] = df['listed_in(type)'].str.strip()

print("Invalid and inconsistent values handled.")

```

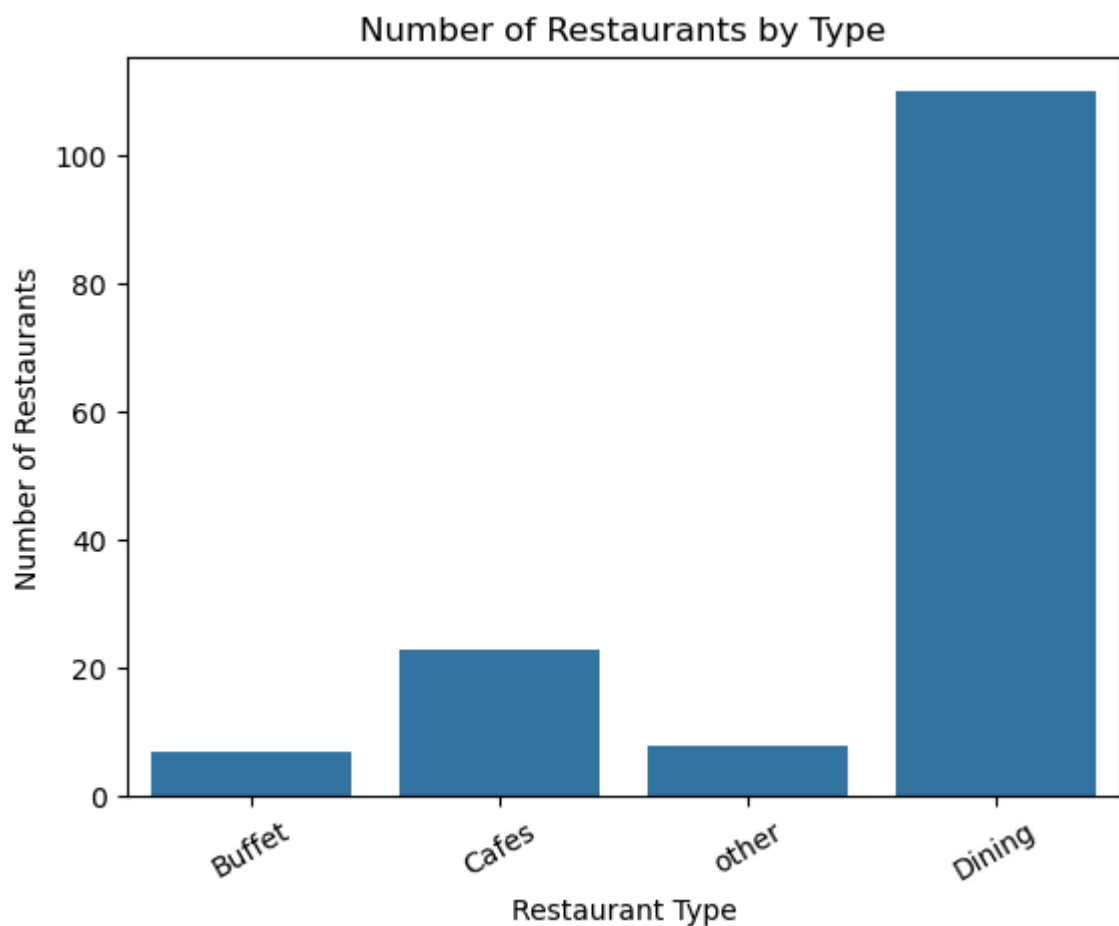
Invalid and inconsistent values handled.

In [52]: `if 'reviews_list' in df.columns:`
`df['reviews_list'] = (`
`df['reviews_list']`
`.astype(str)`
`.str.replace(r'^A-Za-z0-9\s', '', regex=True)`
`)`

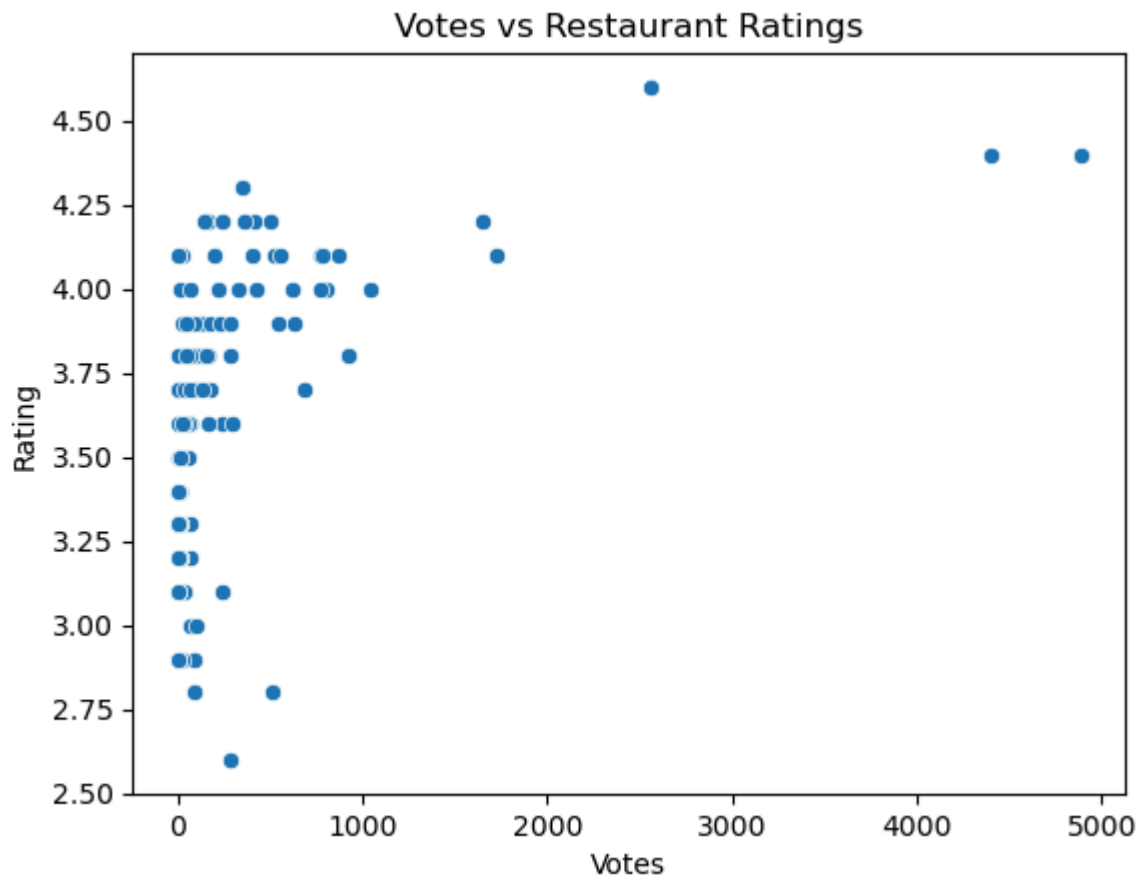
```
.str.replace(r'\s+', ' ', regex=True)
.str.strip()
)
print(df['reviews_list'].head())
else:
    print("Customer review column is not available in this dataset.")
```

Customer review column is not available in this dataset.

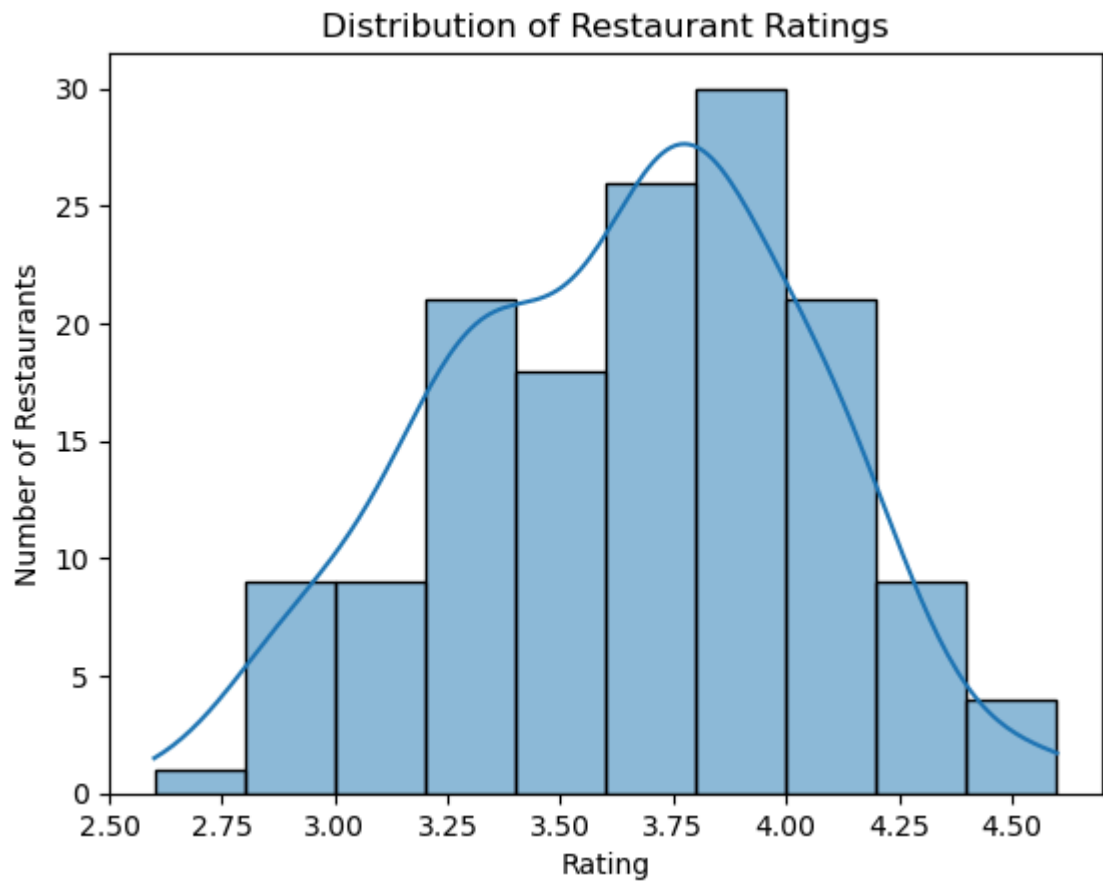
```
In [53]: sns.countplot(data=df, x='listed_in(type)')
plt.title('Number of Restaurants by Type')
plt.xlabel('Restaurant Type')
plt.ylabel('Number of Restaurants')
plt.xticks(rotation=30)
plt.show()
```



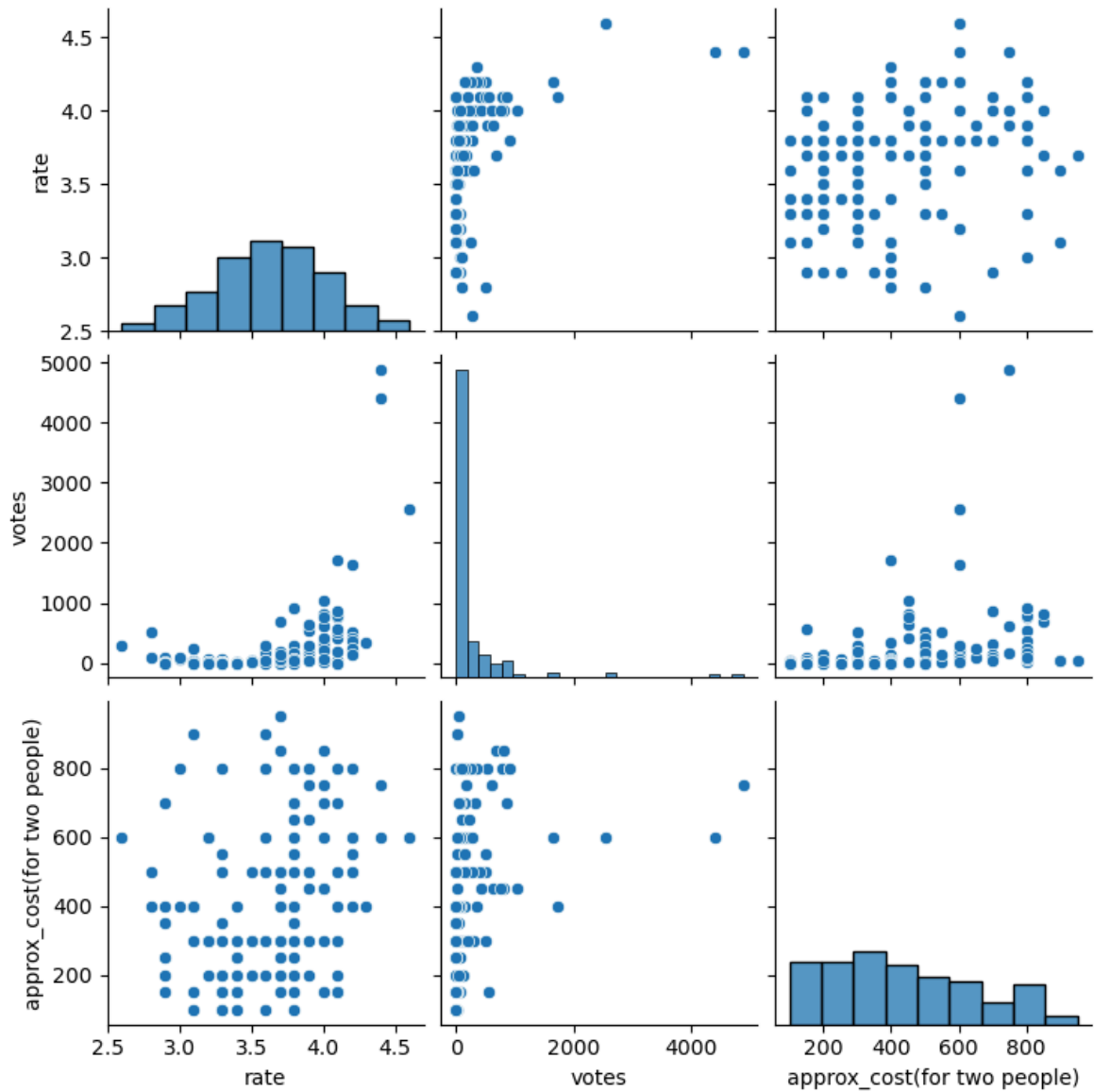
```
In [54]: sns.scatterplot(data=df, x='votes', y='rate')
plt.title('Votes vs Restaurant Ratings')
plt.xlabel('Votes')
plt.ylabel('Rating')
plt.show()
```



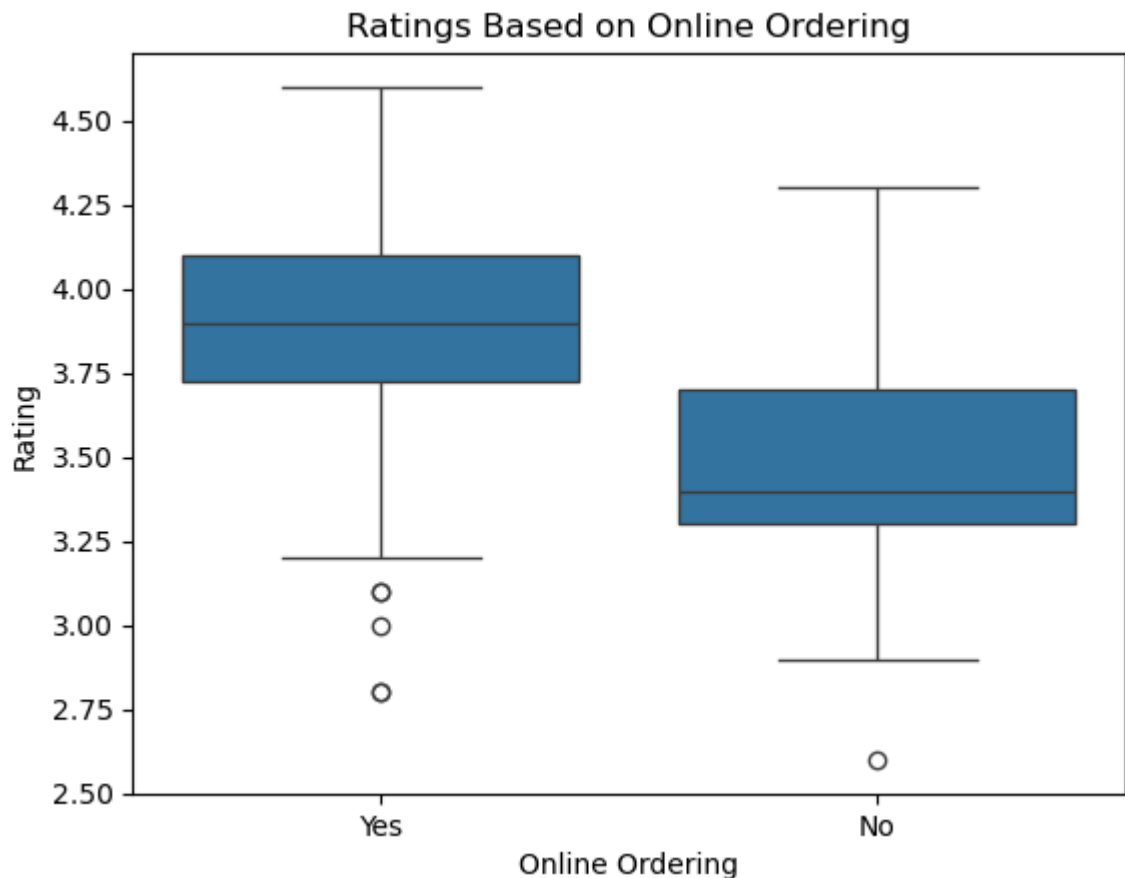
```
In [55]: sns.histplot(df['rate'], bins=10, kde=True)
plt.title('Distribution of Restaurant Ratings')
plt.xlabel('Rating')
plt.ylabel('Number of Restaurants')
plt.show()
```



```
In [56]: sns.pairplot(  
    df[['rate', 'votes', 'approx_cost(for two people)']].dropna()  
)  
plt.show()
```



```
In [57]: sns.boxplot(data=df, x='online_order', y='rate')
plt.title('Ratings Based on Online Ordering')
plt.xlabel('Online Ordering')
plt.ylabel('Rating')
plt.show()
```



```
In [58]: if 'reviews_list' in df.columns:
          from collections import Counter
          import re

          text = ' '.join(df['reviews_list'].astype(str))
          words = re.findall(r'\b[a-zA-Z]+\b', text.lower())

          common_words = Counter(words).most_common(15)
          print(common_words)
        else:
          print("Customer review column is not available.")
```

Customer review column is not available.

```
In [59]: if 'reviews_list' in df.columns:
          from textblob import TextBlob

          def sentiment(text):
              score = TextBlob(text).sentiment.polarity
              if score > 0:
                  return 'Positive'
              elif score < 0:
                  return 'Negative'
              else:
                  return 'Neutral'

          df['sentiment'] = df['reviews_list'].apply(sentiment)

          print(df['sentiment'].value_counts())
        else:
          print("Customer review column is not available.")
```

Customer review column is not available.

```
In [60]: if 'reviews_list' in df.columns:
          print(df.groupby('sentiment')['rate'].mean())

          sns.boxplot(data=df, x='sentiment', y='rate')
          plt.title('Customer Reviews vs Restaurant Ratings')
          plt.xlabel('Review Sentiment')
          plt.ylabel('Rating')
          plt.show()
        else:
          print("Customer review column is not available.")
```

Customer review column is not available.